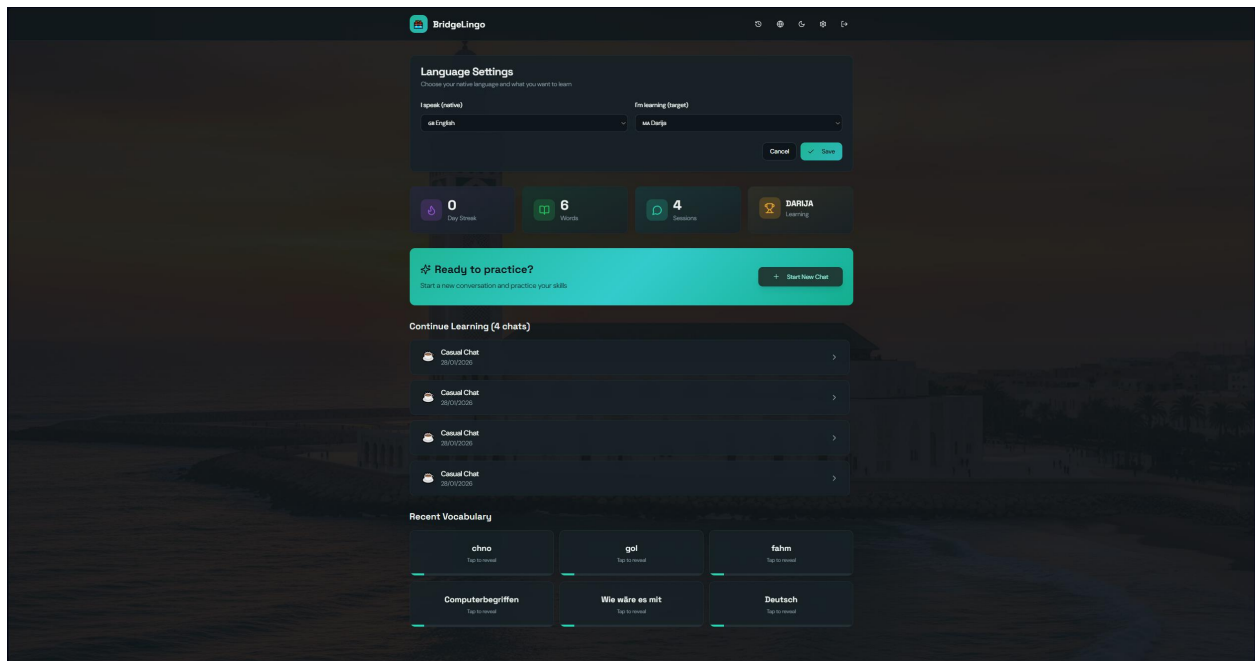


BridgeLingo

Project Report: AI-Powered Language Learning Platform



Marouane AABIRROUCHE
Infrastructure Digitale Student (IDOSR)

January 28, 2026

Executive Summary

BridgeLingo is a personal project developed to explore the intersection of Generative AI and modern web architecture. As a Network Systems student, my goal was to step outside my comfort zone of infrastructure and build a user-facing application that solves a real problem: language immersion.

The platform uses a Retrieval-Augmented Generation (RAG) workflow to support "Code-Switching"—allowing users to mix their native language (like Darija or English) with their target language, while the AI bridges the gaps in real-time.

System Architecture

Leveraging a serverless-first approach, I designed a stack that minimizes maintenance while maximizing performance.

Layer	Technology Stack
Frontend	React 18, TypeScript, Vite
UI Library	Tailwind CSS, shadcn/ui
State	TanStack Query (Server State Management)
Backend	Supabase (PostgreSQL, Auth, Edge Functions)
AI Engine	Google Gemini 3 Flash (via Lovable AI Gateway)
Deployment	Vercel Edge Network

Table 1: Project Tech Stack

Core Logic: The "Code-Switching" Engine

The key innovation of this project is the AI pipeline. I engineered the system prompt to handle "mixed input" naturally:

The Logic Flow

1. **Input:** User mixes languages (e.g., *"I want to buy some **khobz**"*).

2. **Detection:** The AI identifies "khobz" as the bridge word (Darija for bread).

3. **Response:**

• **Correction:** "The word you are looking for is **Brot** (German)."

• **Continuation:** "Möchten Sie Weißbrot oder Schwarzbrot?"

Key Features Implemented

1. Adaptive AI Personas

I implemented a toggle for "System Prompts" to change the AI's behavior:

- **Survival Mode:** High-pressure roleplay (e.g., impatient waiter).
- **Strict Tutor:** Prioritizes grammar correction.
- **Casual Friend:** Optimizes for flow and confidence.

2. Advanced Dialectal Support (Darija)

One of my main goals was to support my native dialect. The AI correctly parses ****Arabizi**** (numeric chat alphabet):

- **Logic:** '3' = ε , '7' = $\hbar$, '9' = q .
- **Result:** It understands idioms like *"Wch a 3chiri"* seamlessly.

3. Immersive Visual Environment ("Vibes")

To enhance the user experience (UX), I created a dynamic theming engine:

- **Berlin Vibe:** Amber tones with Brandenburg Gate background.
- **Casablanca Vibe:** Teal/Terracotta tones with Hassan II Mosque background.

4. Global Accessibility (i18n)

- Full UI translation (English, French, Spanish, German, Arabic).
- ****RTL Support:**** Native Right-to-Left layout for Arabic users.

5. Gamification Retention

- ****Vocabulary Bank:**** Automatically extracts and saves learned words.
- ****Streak Tracking:**** Visual counters to encourage daily practice.

Database Design

Coming from a systems background, structured data was a priority. I used PostgreSQL to map the relationships.

Listing 1: PostgreSQL Schema

```
-- User Profiles & Gamification
CREATE TABLE profiles (
  id UUID PRIMARY KEY,
```

```

    native_lang TEXT NOT NULL,
    target_lang TEXT NOT NULL,
    streak_count INTEGER DEFAULT 0
);

-- Message History (Persistent Storage)
CREATE TABLE messages (
    id UUID PRIMARY KEY,
    conversation_id UUID REFERENCES conversations(id),
    content TEXT NOT NULL,
    role TEXT CHECK (role IN ('user', 'ai')),
    correction_metadata JSONB -- Stores grammar fixes
);

-- Vocabulary Tracking
CREATE TABLE vocabulary_bank (
    id UUID PRIMARY KEY,
    user_id UUID REFERENCES profiles(id),
    word TEXT NOT NULL,
    times_practiced INTEGER DEFAULT 1
);

```

Reflection: The “Vibe Coding” Approach

This project was a personal experiment in **“AI-Assisted Development”** (Vibe Coding).

As a student specializing in **Infrastructure and Networks**, I don’t write React code every day. However, by using AI tools like Lovable and Gemini, I was able to:

- **Bridge the Skill Gap:** I focused on the **logic** and **architecture** (Database schemas, API flows) while the AI handled the syntax boilerplate.
- **Learn by Building:** I learned more about State Management and Vector Databases in one week of building this than I would have in months of reading.
- **Focus on Value:** It allowed me to spend time on the unique features—like the Darija support and the “Vibe” themes—rather than debugging CSS.

BridgeLingo proves that with the right architectural mindset, you can build full-stack applications regardless of your primary specialization.